**1. Introduction.** This is a *literate program*: one file that is both the documentation and the source of a working C program. The technique and the tools are Knuth and Levy's `CWEB`. Two programs read this file. `ctangle` extracts the C code and writes it out in the order a compiler needs; `cweave` extracts the same material and typesets it, with cross-references and an index, in the order a reader needs. The two orders are different, and being able to separate them is the entire point.

The program is *shavar*, a reference implementation of SHA-256. The algorithm is exactly the one specified in the U.S. Federal Information Processing Standard FIPS 180-4; nothing here changes what the function computes. What is different is how it is written down. The standard presents the compression function as eight working registers that shuffle on every round. This program presents it as two coupled recurrences with a lookback of four, which is the subject of §7 below.

**2.** Terms used throughout, defined here before they are used.

- A *word* is a 32-bit unsigned integer. Every addition written + in this document is addition modulo $2^{32}$, which is what C's unsigned arithmetic does by definition.
- A *block* is 512 bits, or 64 bytes.
- The *chaining value* is the eight-word state carried from one block to the next; the digest is its final serialisation.
- A *digest* is the 256-bit output, printed as 64 hexadecimal digits.
- The *message schedule* $W[0 \ldots 63]$ is the sequence of 64 words derived from one block and consumed one per round.
- A *trace* is the complete record of one block's compression: every intermediate word, addressable after the fact.
- GF(2) is the two-element field. An operation is GF(2)-*linear* when it commutes with exclusive-or; this matters in ⟨ The six round functions 26 ⟩.
- *UBSan* is the compiler's UndefinedBehaviorSanitizer; *CAVP* is the NIST Cryptographic Algorithm Validation Program, the source of the known-answer vectors this program is tested against.

**3.** This is not the only presentation of the program in its repository. The directory `c/` holds the same code as ordinary C source files — `shavar.c`, `shavar.h` and `main.c` — and those files are what the rest of the project builds against. This document is a parallel presentation, not a replacement, and it carries an obligation: the program `ctangle` produces from it must be the same program. That obligation is discharged by testing rather than by assertion, and §64 says exactly how.

The order of exposition below is the order in which the algorithm is best learned, which is close to the reverse of the order in which C requires it to be declared:

1. §4, with no code at all: the eight-register form, the two-dimensional form, why they are the same, the message schedule, and padding for a message whose length is not a whole number of bytes.
2. §13: the three files the code is tangled into, and the skeleton of each.
3. §18, ⟨ Word primitives 23 ⟩ and ⟨ The six round functions 26 ⟩: the small pieces.
4. ⟨ The compression function 27 ⟩, ⟨ Padding 33 ⟩ and ⟨ Hashing 37 ⟩: the algorithm in code.
5. §46: plumbing.
6. §64: what is checked, and by what.

**4. The algorithm.** This chapter contains no code. It states the mathematics that the rest of the document implements, so that the code can be read as a transcription of something already understood rather than as the definition of it.

The normative statement lives in `spec/SPEC.md` in the repository, and the exact command-line encoding in `spec/CLI.md`. Where this document and those disagree, those are right.

**5. Notation.** All arithmetic is on words. Bit order is big-endian throughout, matching FIPS 180-4: within a byte the most significant bit comes first, and within a word the most significant byte comes first. That convention is invisible for whole-byte messages and decisive for the sub-byte lengths of §11.

| | |
|---|---|
| $x \oplus y$ | bitwise exclusive or |
| $x \wedge y,\ x \vee y,\ \neg x$ | bitwise and, or, not |
| $x + y$ | addition modulo $2^{32}$ |
| $\mathrm{ROTR}^n(x)$ | circular right rotation by $n$ |
| $\mathrm{SHR}^n(x)$ | logical right shift by $n$, zero-filled |
| $L$ | message length *in bits*, an arbitrary non-negative integer |

Six auxiliary functions are used. Read $\Sigma$ and $\sigma$ as *Sigma* and *sigma*; they have nothing to do with summation.

$$
\begin{aligned}
\mathrm{Ch}(x,y,z) &= (x \wedge y) \oplus (\neg x \wedge z) \\
\mathrm{Maj}(x,y,z) &= (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z) \\
\Sigma_0(x) &= \mathrm{ROTR}^2(x) \oplus \mathrm{ROTR}^{13}(x) \oplus \mathrm{ROTR}^{22}(x) \\
\Sigma_1(x) &= \mathrm{ROTR}^6(x) \oplus \mathrm{ROTR}^{11}(x) \oplus \mathrm{ROTR}^{25}(x) \\
\sigma_0(x) &= \mathrm{ROTR}^7(x) \oplus \mathrm{ROTR}^{18}(x) \oplus \mathrm{SHR}^3(x) \\
\sigma_1(x) &= \mathrm{ROTR}^{17}(x) \oplus \mathrm{ROTR}^{19}(x) \oplus \mathrm{SHR}^{10}(x)
\end{aligned}
$$

Ch is "choose": bit $i$ of $x$ selects between bit $i$ of $y$ and bit $i$ of $z$. Maj is "majority": bit $i$ is whichever value occurs at least twice among the three inputs. The capital-sigma functions act on the state, the lowercase-sigma functions on the message schedule.

**6. The eight-register form.** FIPS 180-4 keeps eight working variables $a, b, \ldots, h$ and, for $t = 0, \ldots, 63$, performs

$$
\begin{aligned}
T_1 &= h + \Sigma_1(e) + \mathrm{Ch}(e,f,g) + K[t] + W[t] \\
T_2 &= \Sigma_0(a) + \mathrm{Maj}(a,b,c) \\
h &\leftarrow g, \quad g \leftarrow f, \quad f \leftarrow e, \quad e \leftarrow d + T_1 \\
d &\leftarrow c, \quad c \leftarrow b, \quad b \leftarrow a, \quad a \leftarrow T_1 + T_2
\end{aligned}
$$

Six of those eight assignments are pure copies. Only $a$ and $e$ are computed. Everything in the next section follows from taking that observation seriously.

**7. The two-dimensional form.** Define two sequences of words, $A[t]$ and $E[t]$, indexed from $t = -4$, and seed them from the incoming chaining value $H[0 \ldots 7]$:

$$\begin{array}{llll} A[-1] = H[0] & A[-2] = H[1] & A[-3] = H[2] & A[-4] = H[3] \\ E[-1] = H[4] & E[-2] = H[5] & E[-3] = H[6] & E[-4] = H[7] \end{array}$$

Note the reversal: the more negative the index, the later the $H$. Then for $t = 0, \ldots, 63$,

$$\begin{aligned} T_1[t] &= E[t-4] + \Sigma_1(E[t-1]) + \mathrm{Ch}(E[t-1], E[t-2], E[t-3]) + K[t] + W[t] \\ T_2[t] &= \Sigma_0(A[t-1]) + \mathrm{Maj}(A[t-1], A[t-2], A[t-3]) \\ E[t] &= A[t-4] + T_1[t] \\ A[t] &= T_1[t] + T_2[t] \end{aligned}$$

and the outgoing chaining value is

$$\begin{array}{llll} H[0] \mathrel{+}= A[63] & H[1] \mathrel{+}= A[62] & H[2] \mathrel{+}= A[61] & H[3] \mathrel{+}= A[60] \\ H[4] \mathrel{+}= E[63] & H[5] \mathrel{+}= E[62] & H[6] \mathrel{+}= E[61] & H[7] \mathrel{+}= E[60] \end{array}$$

**8. Why the two forms are the same.** The eight registers are not eight independent things. They are two sliding windows of width four, one over the history of $A$ and one over the history of $E$. At the top of round $t$ the correspondence is exactly

$$\begin{array}{llll} a = A[t-1] & b = A[t-2] & c = A[t-3] & d = A[t-4] \\ e = E[t-1] & f = E[t-2] & g = E[t-3] & h = E[t-4] \end{array}$$

Substituting that into the eight-register form turns the six copy-assignments into nothing at all — they are the window sliding — and the two real assignments into the two recurrences above. The register shuffle was never computation; it was an artefact of writing a recurrence with an explicit shift register. This equivalence is machine-checked in Lean in the same repository, in `lean/Shavar/Equiv.lean`; it is not merely asserted here.

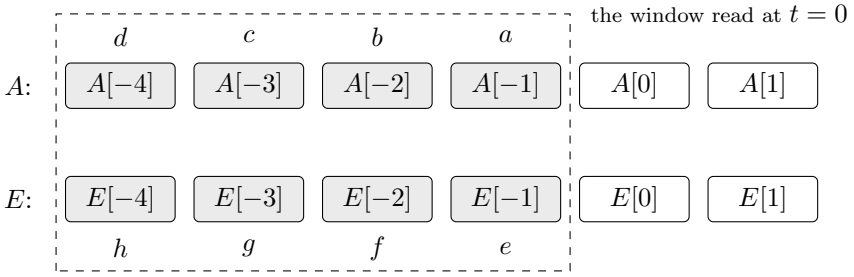


*Figure 1.* The eight working registers of the standard presentation are a width-4 window over two sequences. Rounded boxes are 32-bit words; shaded boxes are the seeds taken from the incoming chaining value. Advancing the round by one slides the dashed window one box to the right, which is what the six copy-assignments were doing.

**9. Dependency depth, and the asymmetry between the tracks.** $A[t]$ depends on $A[t-1\ldots t-4]$ and on $E[t-1\ldots t-4]$; $E[t]$ depends on $E[t-1\ldots t-4]$ and on $A[t-4]$. Both recurrences are order 4.

One asymmetry is worth remembering because it is invisible in the eight-register form. The $E$ track reaches into the $A$ history at exactly one point, $A[t-4]$, whereas the $A$ track reaches into the $E$ history at four points. That single term is the only path by which $A$ influences $E$ at all.

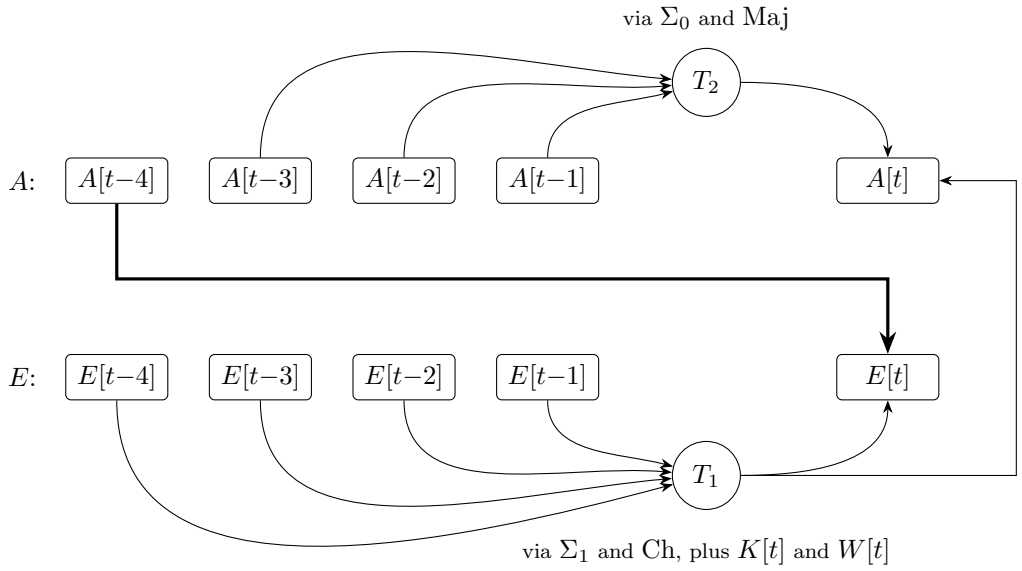


*Figure 2.* One round of the recurrence. Rounded boxes are 32-bit words of state; circles are the two accumulated sums $T_1$ and $T_2$; every arrow is one 32-bit word flowing into a computation. The heavy arrow is the single term $A[t-4]$ entering $E[t]$, which is the only influence the $A$ track has on the $E$ track — compare the four arrows the $A$ track draws from the $E$ track. The round functions are applied on the arrows entering $T_1$ and $T_2$ and are named beside those nodes rather than drawn, so that the picture shows the structure of the recurrence and not the arithmetic.

**10. The message schedule.** The schedule has the same shape one dimension down. Split a 512-bit block into sixteen big-endian words $M[0\ldots15]$; then

$$W[t] = M[t] \qquad 0 \le t < 16$$
$$W[t] = \sigma_1(W[t-2]) + W[t-7] + \sigma_0(W[t-15]) + W[t-16] \qquad 16 \le t < 64$$

So the whole of SHA-256 is one order-16 recurrence ($W$) driving a nonlinear order-4 recurrence in two tracks ($A$ and $E$). That two-clause sentence is the entire algorithm.

$W$ is worth isolating because it is GF(2)-linear apart from its three additions: $\sigma_0$ and $\sigma_1$ are exclusive-ors of rotations and shifts, and only the carries in $+$ break linearity. Message-modification attacks exploit precisely this.

**11. Padding for an arbitrary number of bits.** $L$ is a bit count, not a byte count, and may be any non-negative integer, including one that is not a multiple of 8. This is required by FIPS 180-4 and is where most casual implementations quietly do the wrong thing. Append to the message:

1. a single 1 bit;
2. $k$ zero bits, where $k$ is the smallest non-negative solution of $L + 1 + k \equiv 448 \pmod{512}$;
3. $L$ itself as a 64-bit big-endian integer.

The result has length $L + 1 + k + 64 \equiv 0 \pmod{512}$.

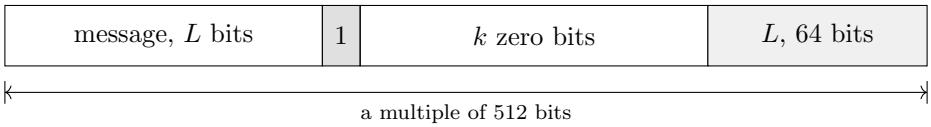


*Figure 3.* The padded message. The 64-bit length field at the end is what makes padding injective, which the Merkle–Damgård security argument requires: without it two distinct messages could pad to the same bit string and collide for reasons having nothing to do with the compression function. Injectivity is proved in Lean in `lean/` rather than argued here.

**12. The representation convention, and why nonzero trailing bits are rejected.** A message is carried as a byte buffer plus a bit count $L$. The buffer holds $\lceil L/8 \rceil$ bytes. When $L$ is not a multiple of 8 the final byte holds its $L \bmod 8$ significant bits *in the high-order positions*, and the remaining low-order bits of that byte must be zero.

An implementation that met a nonzero low bit could do one of two things: mask it away, or refuse the input. This one refuses. Masking would map two distinct inputs to a single digest and would hide the caller's bug; refusing costs one comparison and reports it. The choice is normative in `spec/SPEC.md` and is implemented in ⟨ Padding 33 ⟩.

Worked example, for $L = 5$ and the five bits 10110: the buffer is the one byte $10110000_2$ = `0xB0`; the appended 1 bit lands at position 5, giving $10110100_2$ = `0xB4`; zeros then fill to bit 448 and the 64-bit value 5 ends the block. Note that `0xB4` is exactly the byte that must be *rejected* when it is offered as a 5-bit message, since its low three bits are 100.

$L = 0$ is legal: the padded message is a single block of `0x80` followed by 63 zero bytes.

**13. The shape of the program.** `ctangle` writes three files from this one. Their names are chosen so that the tangled program is a drop-in replacement for the hand-written one in `c/`, which is what makes the equivalence test of §64 possible.

- `shavar-cweb.c` — the library. This is the unnamed section of the web, so it goes to the default output file, whose name follows the name of this file.
- `shavar.h` — the public interface, ⟨ `shavar.h` 18 ⟩, described in §18.
- `main.c` — the command-line driver, ⟨ `main.c` 46 ⟩, described in §46.

The library file contains no input or output and no mutable global state, so it is reentrant, thread-safe without locking, and linkable into a freestanding environment. All of the input and output lives in the driver.

**14. Portability stance.** The code is strict C99 with no compiler extensions, no POSIX, and no libraries beyond `<stdint.h>`, `<stddef.h>` and `<string.h>`. Three specific commitments follow from that, and each is visible in the code:

- *No dependence on host endianness.* Every conversion between bytes and words is written as explicit shifts of individual bytes. That is why no cast from **unsigned char** pointer to **uint32_t** pointer appears anywhere; such a cast would be both endian-dependent and an alignment violation. The repository's continuous integration cross-compiles this file for s390x and runs it under emulation, so the claim is tested on a big-endian machine rather than merely stated.
- *No dependence on signed-integer behaviour.* Everything is unsigned, so there is no overflow undefined behaviour: unsigned arithmetic wraps by definition in C, which is exactly the modulo-$2^{32}$ semantics the algorithm wants.
- *No dynamic allocation.* There is not one call to an allocator in the library or in the driver. All state is caller-provided or automatic, which makes the memory-safety question trivially answerable.

**15.** Here is the whole library file. Each name in angle brackets is a section defined later; the number after the name is the section that defines it, and in the on-screen version it is a link. Reading this section top to bottom gives the order the C compiler sees, which is the only thing that order has to satisfy.

⟨ Library inclusions 16 ⟩
⟨ The constant tables 22 ⟩
⟨ Word primitives 23 ⟩
⟨ The six round functions 26 ⟩
⟨ The compression function 27 ⟩
⟨ Padding 33 ⟩
⟨ Hashing 37 ⟩
⟨ Proof of work 42 ⟩

**16.** ⟨ Library inclusions 16 ⟩ ≡

```
#include "shavar.h"
#include <string.h>
```

This code is used in section 15.

**17.** Type names imported from `<stdint.h>` and `<stddef.h>` are declared to `cweave` here so that it sets them as types rather than as variables. This affects only the typeset output; `ctangle` ignores it.

**format** *uint32_t int*
**format** *uint64_t int*
**format** *size_t int*

**18. The public interface.** The header is the contract that the other six implementations in the repository mirror, and against which they are tested round by round. It is deliberately small.

⟨ shavar.h 18 ⟩ ≡
```
#ifndef SHAVAR_H
#define SHAVAR_H
#include <stddef.h>
#include <stdint.h>
  ⟨ Interface constants 19 ⟩
  ⟨ The trace structure 20 ⟩
  ⟨ Interface declarations 21 ⟩
#endif
```
This code is cited in section 13.

**19.** SHAVAR_ROUNDS is 64 for full SHA-256. It is a compile-time bound on the arrays below rather than the number of rounds actually run: the run-time round count is a parameter, because reduced-round variants are one of the two things this library exists to make expressible. The other is a caller-chosen initial chaining value.

⟨ Interface constants 19 ⟩ ≡
```
#define SHAVAR_ROUNDS 64
#define SHAVAR_DIGEST_BYTES 32
#define SHAVAR_BLOCK_BYTES 64
```
This code is used in section 18.

**20.** The trace is a complete record of one block's compression.

Its shape is a direct consequence of §7. In the eight-register presentation the state is eight words and recording the history would be extra work; here the history *is* the state, so retaining the whole trajectory costs nothing. That is 544 bytes of $A$ and $E$ history per block, and it is what makes every intermediate value addressable after the fact — which is what differential and algebraic analysis of the function actually needs.

The $A$ and $E$ arrays are offset by 4 so that index $4+t$ holds element $t$ of the mathematical sequence and $t$ may run from $-4$. Callers should use the two accessors rather than open-coding the $+4$, which is the obvious place to introduce an off-by-one.

⟨ The trace structure 20 ⟩ ≡

```
typedef struct {
  uint32_t A[4 + SHAVAR_ROUNDS];
  uint32_t E[4 + SHAVAR_ROUNDS];
  uint32_t W[SHAVAR_ROUNDS];
  uint32_t T1[SHAVAR_ROUNDS];
  uint32_t T2[SHAVAR_ROUNDS];
  uint32_t h_in[8];
  uint32_t h_out[8];
  int rounds;
} shavar_trace;
static inline uint32_t shavar_A(const shavar_trace *tr, int t)
{
  return tr->A[4 + t];
}
static inline uint32_t shavar_E(const shavar_trace *tr, int t)
{
  return tr->E[4 + t];
}
```

This code is cited in section 27.

This code is used in section 18.

**21.** The declarations themselves. Four groups: the constant tables, the six round functions, the compression function, and the hashing entry points.

The six round functions are exported individually rather than fused into the round body. The reason is the GF(2) split described in ⟨ The six round functions 26 ⟩: research code needs to instrument or substitute them one at a time, which requires that they exist as separate, callable, externally visible functions.

*shavar_hash* hashes *nbits* bits of *msg*, which must hold $\lceil nbits/8 \rceil$ bytes, and returns 0 on success or $-1$ if the final byte has nonzero trailing bits. *shavar_hash_ex* is the same from a caller-chosen initial chaining value and round count. *shavar_padded_blocks* and *shavar_padded_block* let a caller walk the padded stream one block at a time without ever materialising it.

*shavar_pow_target* and *shavar_pow_check* are the proof-of-work comparison of §40. They are library-only: no subcommand of the driver reaches them, because the uniform command-line contract is shared with six other implementations and is not the place to grow a verb that only some callers want.

⟨ Interface declarations 21 ⟩ ≡

```
extern const uint32_t shavar_iv[8];
extern const uint32_t shavar_k[64];
uint32_t shavar_ch(uint32_t x, uint32_t y, uint32_t z);
uint32_t shavar_maj(uint32_t x, uint32_t y, uint32_t z);
uint32_t shavar_Sigma0(uint32_t x);
uint32_t shavar_Sigma1(uint32_t x);
uint32_t shavar_sigma0(uint32_t x);
uint32_t shavar_sigma1(uint32_t x);
void shavar_compress(uint32_t h[8], const unsigned char block[64], int rounds, shavar_trace *tr);
int shavar_hash(const unsigned char *msg, uint64_t nbits,
    unsigned char out[SHAVAR_DIGEST_BYTES]);
int shavar_hash_ex(const unsigned char *msg, uint64_t nbits, const uint32_t iv[8], int
    rounds, unsigned char out[SHAVAR_DIGEST_BYTES]);
uint64_t shavar_padded_blocks(uint64_t nbits);
int shavar_padded_block(const unsigned char *msg, uint64_t nbits, uint64_t idx, unsigned char
    out[64]);
int shavar_pow_target(uint32_t nbits, unsigned char target[SHAVAR_DIGEST_BYTES]);
int shavar_pow_check(const unsigned char digest[SHAVAR_DIGEST_BYTES], uint32_t nbits);
```

This code is used in section 18.

**22. The constant tables.** The initial chaining value $H[0\ldots7]$ is the first 32 bits of the fractional parts of the square roots of the first eight primes. The round constants $K[0\ldots63]$ are the first 32 bits of the fractional parts of the cube roots of the first sixty-four primes.

Both derivations are checkable, and the repository's test harness recomputes them from the primes in exact integer arithmetic rather than trusting the transcription below. A mistyped constant is the classic way one implementation of several ends up subtly different from the rest, and it is cheap to rule out. Note that $K$ is not observable through the command-line interface at all: a mistyped $K[t]$ surfaces as a trace divergence at $T_1[t]$, which is what the cross-implementation trace diff reports.

⟨ The constant tables 22 ⟩ ≡

```
const uint32_t shavar_iv[8] = {0x6a09e667U, 0xbb67ae85U, 0x3c6ef372U, 0xa54ff53aU, 0x510e527fU,
    0x9b05688cU, 0x1f83d9abU, 0x5be0cd19U};
const uint32_t shavar_k[64] = {0x428a2f98U, 0x71374491U, 0xb5c0fbcfU, 0xe9b5dba5U, 0x3956c25bU,
    0x59f111f1U, 0x923f82a4U, 0xab1c5ed5U, 0xd807aa98U, 0x12835b01U, 0x243185beU, 0x550c7dc3U,
    0x72be5d74U, 0x80deb1feU, 0x9bdc06a7U, 0xc19bf174U, 0xe49b69c1U, 0xefbe4786U, 0x0fc19dc6U,
    0x240ca1ccU, 0x2de92c6fU, 0x4a7484aaU, 0x5cb0a9dcU, 0x76f988daU, 0x983e5152U, 0xa831c66dU,
    0xb00327c8U, 0xbf597fc7U, 0xc6e00bf3U, 0xd5a79147U, 0x06ca6351U, 0x14292967U, 0x27b70a85U,
    0x2e1b2138U, 0x4d2c6dfcU, 0x53380d13U, 0x650a7354U, 0x766a0abbU, 0x81c2c92eU, 0x92722c85U,
    0xa2bfe8a1U, 0xa81a664bU, 0xc24b8b70U, 0xc76c51a3U, 0xd192e819U, 0xd6990624U, 0xf40e3585U,
    0x106aa070U, 0x19a4c116U, 0x1e376c08U, 0x2748774cU, 0x34b0bcb5U, 0x391c0cb3U, 0x4ed8aa4aU,
    0x5b9cca4fU, 0x682e6ff3U, 0x748f82eeU, 0x78a5636fU, 0x84c87814U, 0x8cc70208U, 0x90befffaU,
    0xa4506cebU, 0xbef9a3f7U, 0xc67178f2U};
```

This code is used in section 15.

**23. Word primitives.** Two helpers, a rotation and a shift, plus a mask. `M32` looks redundant when **uint32_t** is **unsigned int**, and in that common case it is. It is not redundant in general: on a platform with 64-bit **int**, **uint32_t** would be promoted to signed **int** by C's integer promotions, and a left shift could then set bits above position 31. Masking pins every result to 32 bits regardless of how the promotions land.

```
⟨ Word primitives 23 ⟩ ≡
#define M32 ^FFFFFFFF_U
  ⟨ Rotate right 24 ⟩
  ⟨ Shift right 25 ⟩
```

This code is cited in sections 3 and 52.

This code is used in section 15.

**24.** The rotation is the one place in this program where the obvious code was rewritten for a reason worth recording.

The classic idiom for a rotation is the bitwise or of $x \gg n$ and $x \ll (32 - n)$. That idiom is perfectly well defined: C99 6.5.7p4 defines the left shift of an unsigned type as reduction modulo $2^{32}$, so discarding the high bits is specified behaviour, not undefined behaviour. There is no bug in it.

UBSan nevertheless has a check, `unsigned-shift-base`, that flags *any* left shift which discards bits as suspicious — and a rotation discards bits by design. The classic idiom therefore produces a permanent false positive, which would have to be silenced by a suppression file or by excluding that one check from the sanitizer configuration.

The version below masks off the low $n$ bits *before* shifting them up, so no bit is ever shifted out of the top of the word and the check has nothing to complain about. The whole sanitizer matrix then runs with no suppressions and no exclusions to explain away. Two things make this a good trade rather than a superstitious one: the code is no less correct than the idiom it replaces, and at `-O2` the compiler emits the same single rotate instruction for both, which was verified in the disassembly rather than assumed.

The guard on $n \equiv 0$ is a second, independent point. $x \ll (32 - n)$ is undefined when $n$ is zero, because a shift by the full width of the type is undefined in C. None of SHA-256's rotation amounts are zero, so the guard is unreachable in this program; it is written anyway so that the helper is correct in isolation rather than only correct for the callers it happens to have.

```
⟨ Rotate right 24 ⟩ ≡
  static uint32_t rotr(uint32_t x, unsigned n)
  {
    n &= 31_U;
    if (n ≡ 0_U) return x;
    return ((x ≫ n) | ((x & ((1_U ≪ n) − 1_U)) ≪ (32_U − n))) & M32;
  }
```

This code is used in section 23.

**25.**

```
⟨ Shift right 25 ⟩ ≡
  static uint32_t shr(uint32_t x, unsigned n)
  {
    return (x ≫ n) & M32;
  }
```

This code is used in section 23.

**26. The six round functions.** These are the functions of the notation section, transcribed. They are not declared **static**, and they are not fused into the round body, for a reason that is about research use rather than about style.

Partition SHA-256's operations by their behaviour over GF(2). Exclusive-or, ROTR and SHR are linear, and therefore so are $\Sigma_0$, $\Sigma_1$, $\sigma_0$ and $\sigma_1$. Addition is nonlinear, through its carries, and so are Ch and Maj. If addition were replaced by exclusive-or and Ch and Maj by linear functions, SHA-256 would collapse into an affine map over $\mathrm{GF}(2)^{256}$ and fall to linear algebra alone. All of its strength sits in the carry chains and in those two bitwise selectors.

A second observation follows from the same partition. Ch, Maj and exclusive-or all act bit-position-wise: bit $i$ of the output depends only on bit $i$ of the inputs. Rotations permute bit positions but do not mix them. The only operation that moves information from bit $i$ to a higher bit $j$ is the carry in an addition. Carry propagation is therefore the sole mechanism of diffusion across the word, and it is directional — carries move towards the most significant bit only. That asymmetry is why low-order bit differences are cheap to control and high-order ones are not, and why published differential paths concentrate their differences in the low bits.

Keeping the six functions separately addressable is what lets a caller substitute or instrument them one at a time. Performance is not a concern here; the compiler inlines them at `-O2` in any case.

⟨ The six round functions 26 ⟩ ≡

```
uint32_t shavar_ch(uint32_t x, uint32_t y, uint32_t z)
{
  return ((x & y) ⊕ (∼x & z)) & M32;
}
uint32_t shavar_maj(uint32_t x, uint32_t y, uint32_t z)
{
  return ((x & y) ⊕ (x & z) ⊕ (y & z)) & M32;
}
uint32_t shavar_Sigma0(uint32_t x)
{
  return rotr(x, 2) ⊕ rotr(x, 13) ⊕ rotr(x, 22);
}
uint32_t shavar_Sigma1(uint32_t x)
{
  return rotr(x, 6) ⊕ rotr(x, 11) ⊕ rotr(x, 25);
}
uint32_t shavar_sigma0(uint32_t x)
{
  return rotr(x, 7) ⊕ rotr(x, 18) ⊕ shr(x, 3);
}
uint32_t shavar_sigma1(uint32_t x)
{
  return rotr(x, 17) ⊕ rotr(x, 19) ⊕ shr(x, 10);
}
```

This code is cited in sections 2, 3, and 21.

This code is used in section 15.

**27. The compression function.** This is the heart of the program: one 64-byte block folded into the eight-word chaining value $h$, with everything recorded into $*tr$.

*tr* may be null when the caller does not want a trace, but the trace is built either way, into a local if necessary. That is not wasted work. As ⟨ The trace structure 20 ⟩ explains, in the two-dimensional form the history *is* the state; there is no cheaper representation to fall back to.

*rounds* may be fewer than 64, and $h$ may be any chaining value rather than *shavar_iv*. Neither is reachable through a hashing API and both are prerequisites for free-start and reduced-round analysis.

⟨ The compression function 27 ⟩ ≡

```
void shavar_compress(uint32_t h[8], const unsigned char block[64], int rounds, shavar_trace *tr)
{
  shavar_trace local;
  shavar_trace *s = tr ? tr : &local;
  int t;
  ⟨ Clamp the round count and record the incoming chaining value 28 ⟩
  ⟨ Build the message schedule 29 ⟩
  ⟨ Seed the two tracks 30 ⟩
  ⟨ Run the recurrence 31 ⟩
  ⟨ Feed forward 32 ⟩
  for (t = 0; t < 8; t++) s->h_out[t] = h[t];
}
```

This code is cited in sections 3 and 52.

This code is used in section 15.

**28.** The clamp here is defensive, not the specification of the valid range. The command-line contract requires that a round count outside 0 . . . 64 be *rejected* with a diagnostic, not clamped, and that rejection happens in ⟨ Parsing the round count 52 ⟩ before this function is ever called. The two behaviours differ in exactly the case that matters: clamping makes a request that means nothing produce a correct-looking SHA-256 digest.

Zero rounds is legal and is not the same as an error. It is the degenerate case in which no round runs and the block contributes only its feed-forward, which isolates the seeding and feed-forward logic from the round function entirely.

⟨ Clamp the round count and record the incoming chaining value 28 ⟩ ≡

```
if (rounds < 0) rounds = 0;
if (rounds > SHAVAR_ROUNDS) rounds = SHAVAR_ROUNDS;
s->rounds = rounds;
for (t = 0; t < 8; t++) s->h_in[t] = h[t];
```

This code is used in section 27.

**29.** The first sixteen words come straight from the block, big-endian, written as explicit byte shifts for the endianness reason given in §13. The remaining forty-eight are generated by the order-16 recurrence.

The second loop always runs to 64, even for a reduced-round compression. The schedule is defined independently of how many rounds consume it, and the command-line contract requires all 64 entries to be reported by `trace` whatever the round count is.

```
⟨ Build the message schedule 29 ⟩ ≡
  for (t = 0; t < 16; t++) {
    s→W[t] = ((uint32_t) block[4 * t + 0] ≪ 24) | ((uint32_t) block[4 * t + 1] ≪ 16) |
        ((uint32_t) block[4 * t + 2] ≪ 8) | ((uint32_t) block[4 * t + 3]);
  }
  for (t = 16; t < SHAVAR_ROUNDS; t++) {
    s→W[t] = (shavar_sigma1(s→W[t − 2]) + s→W[t − 7] + shavar_sigma0(s→W[t − 15]) + s→W[t − 16]) & M32;
  }
```

This code is cited in section 55.

This code is used in section 27.

**30.** Seeding, with the reversal of §7: the more negative the index, the later the $H$. Writing $s{\to}A[4-1]$ rather than $s{\to}A[3]$ keeps the correspondence with $A[-1]$ visible at the point of use. Getting this backwards produces wrong digests immediately, which is the good kind of bug.

```
⟨ Seed the two tracks 30 ⟩ ≡
  s→A[4 − 1] = h[0];
  s→A[4 − 2] = h[1];
  s→A[4 − 3] = h[2];
  s→A[4 − 4] = h[3];
  s→E[4 − 1] = h[4];
  s→E[4 − 2] = h[5];
  s→E[4 − 3] = h[6];
  s→E[4 − 4] = h[7];
```

This code is used in section 27.

**31.** The recurrence itself: four lines, a transcription of the four equations in §7. There is no register shuffle because there are no registers to shuffle.

The local pointers $A$ and $E$ are offset by 4 so that $A[t-4]$ is legal C for $t \geq 0$ and reads the way the equation does. They are recomputed on each iteration, which the compiler removes; writing them inside the loop keeps their scope as small as the use.

```
⟨ Run the recurrence 31 ⟩ ≡
  for (t = 0; t < rounds; t++) {
    uint32_t *A = s→A + 4;
    uint32_t *E = s→E + 4;
    uint32_t t1 = (E[t − 4] + shavar_Sigma1(E[t − 1]) + shavar_ch(E[t − 1], E[t − 2],
        E[t − 3]) + shavar_k[t] + s→W[t]) & M32;
    uint32_t t2 = (shavar_Sigma0(A[t − 1]) + shavar_maj(A[t − 1], A[t − 2], A[t − 3])) & M32;
    E[t] = (A[t − 4] + t1) & M32;
    A[t] = (t1 + t2) & M32;
    s→T1[t] = t1;
    s→T2[t] = t2;
  }
```

This code is used in section 27.

**32.** The feed-forward adds the final window of each track into the incoming chaining value. For the full 64 rounds that window is $A[63\ldots60]$ and $E[63\ldots60]$.

When *rounds* is smaller the window is the last four computed values, and for fewer than four rounds it reaches back into the seeds. That case needs no special handling here, because the seeds live in the same array at the negative indices, so $A[r-4]$ addresses the right word whatever $r$ is. This is a small dividend of the offset-by-4 representation.

⟨ Feed forward 32 ⟩ ≡

```
{
  uint32_t *A = s->A + 4;
  uint32_t *E = s->E + 4;
  int r = rounds;
  h[0] = (h[0] + A[r - 1]) & M32;
  h[1] = (h[1] + A[r - 2]) & M32;
  h[2] = (h[2] + A[r - 3]) & M32;
  h[3] = (h[3] + A[r - 4]) & M32;
  h[4] = (h[4] + E[r - 1]) & M32;
  h[5] = (h[5] + E[r - 2]) & M32;
  h[6] = (h[6] + E[r - 3]) & M32;
  h[7] = (h[7] + E[r - 4]) & M32;
}
```

This code is used in section 27.

**33. Padding.** Two functions implement §11. The pair is deliberately shaped so that the padded message is never materialised: a caller asks how many blocks there are, then asks for each one in turn.

```
⟨ Padding 33 ⟩ ≡
  ⟨ Count the padded blocks 34 ⟩
  ⟨ Produce one padded block 35 ⟩
```

This code is cited in sections 3 and 12.

This code is used in section 15.

**34.** The block count is the smallest multiple of 512 that is at least $L + 1 + 64$, divided by 512.

The obvious expression is $(nbits + 576)/512$, and it is wrong: it overflows for *nbits* near the top of the 64-bit range. The bit length is allowed to be any 64-bit value, so the arithmetic has to survive the top of the range even though no real message reaches it. Splitting into quotient and remainder first keeps every intermediate below 1088.

```
⟨ Count the padded blocks 34 ⟩ ≡
  uint64_t shavar_padded_blocks(uint64_t nbits)
  {
    uint64_t q = nbits/512U;
    uint64_t r = nbits % 512U;
    return q + (r + 576U)/512U;
  }
```

This code is used in section 33.

**35.** Producing block number *idx* of the padded message. *full_bytes* is the count of bytes that are wholly message, and *partial* is the number of significant bits in the byte after those, which is zero when the message ends on a byte boundary.

The function returns −1 in two cases: when *idx* is past the end, and when the final byte carries nonzero trailing bits. The second is the rejection rule argued for in §11. Because that check has already run by the time the byte is emitted below, the padding bit can be combined with an ∨ and no masking is needed — the low bits are known to be zero.

```
⟨ Produce one padded block 35 ⟩ ≡
  int shavar_padded_block(const unsigned char *msg, uint64_t nbits, uint64_t idx, unsigned char
          out[64])
  {
    uint64_t nblocks = shavar_padded_blocks(nbits);
    uint64_t full_bytes = nbits/8U;
    unsigned partial = (unsigned)(nbits % 8U);
    uint64_t base;
    uint64_t last_base;
    int j;
    if (idx ≥ nblocks) return −1;
    if (partial ≠ 0U) {
      unsigned char junk = (unsigned char)(msg[full_bytes] & (#FFU ≫ partial));
      if (junk ≠ 0U) return −1;
    }
    base = idx * 64U;
    last_base = nblocks * 64U − 8U;
    for (j = 0; j < 64; j++) {
      uint64_t gb = base + (uint64_t) j;
      unsigned char v;
      ⟨ Decide the value of one padded byte 36 ⟩
      out[j] = v;
    }
    return 0;
  }
```

This code is used in section 33.

**36.** One byte of the padded stream, selected by its global index $gb$. The four cases are, in the order tested: the 64-bit length field at the very end; a byte that is wholly message; the single byte where the message ends and the padding 1 bit begins; and the zero fill between them.

The third case is the interesting one. If *partial* is zero the message ended on a byte boundary and this byte is purely padding, `0x80`. Otherwise the message supplies the top *partial* bits and the 1 bit goes immediately below them, at $^{\#}\texttt{80}_{\mathrm{U}} \gg \mathit{partial}$.

```
⟨ Decide the value of one padded byte 36 ⟩ ≡
  if (gb ≥ last_base) {
    unsigned shift = (unsigned)(8U * (7U − (gb − last_base)));
    v = (unsigned char)((nbits ≫ shift) & #FFU);
  }
  else if (gb < full_bytes) {
    v = msg[gb];
  }
  else if (gb ≡ full_bytes) {
    v = partial ? (unsigned char)(msg[gb] | (#80U ≫ partial)) : (unsigned char) #80U;
  }
  else {
    v = 0U;
  }
```

This code is used in section 35.

**37. Hashing.** The Merkle–Damgård loop: start from an initial chaining value, compress every padded block in turn, and serialise the result.

```
⟨ Hashing 37 ⟩ ≡
  ⟨ Hash from a given initial value 38 ⟩
  ⟨ Hash from the standard initial value 39 ⟩
```

This code is cited in section 3.

This code is used in section 15.

**38.** Note that no trace is requested from *shavar_compress* here — the last argument is null — because hashing does not need one. The trace path is exercised only by the `trace` subcommand of §46.

The two *memset* calls at the end scrub the working state. This is not offered as a security guarantee: a compiler is permitted to elide a store to an object that is about to die, and a hash of public data does not need scrubbing anyway. They are there because they cost nothing on a 64-byte buffer and they keep the sanitizer runs honest about what remains live.

```
⟨ Hash from a given initial value 38 ⟩ ≡
  int shavar_hash_ex(const unsigned char *msg, uint64_t nbits, const uint32_t iv[8], int
          rounds, unsigned char out[SHAVAR_DIGEST_BYTES])
  {
    uint32_t h[8];
    unsigned char block[64];
    uint64_t nblocks = shavar_padded_blocks(nbits);
    uint64_t i;
    int j;
    for (j = 0; j < 8; j++) h[j] = iv[j];
    for (i = 0; i < nblocks; i++) {
      if (shavar_padded_block(msg, nbits, i, block) ≠ 0) return −1;
      shavar_compress(h, block, rounds, Λ);
    }
    for (j = 0; j < 8; j++) {
      out[4 * j + 0] = (unsigned char)((h[j] ≫ 24) & #FF_U);
      out[4 * j + 1] = (unsigned char)((h[j] ≫ 16) & #FF_U);
      out[4 * j + 2] = (unsigned char)((h[j] ≫ 8) & #FF_U);
      out[4 * j + 3] = (unsigned char)(h[j] & #FF_U);
    }
    memset(block, 0, sizeof block);
    memset(h, 0, sizeof h);
    return 0;
  }
```

This code is used in section 37.

**39.**

```
⟨ Hash from the standard initial value 39 ⟩ ≡
  int shavar_hash(const unsigned char *msg, uint64_t nbits, unsigned char out[SHAVAR_DIGEST_BYTES])
  {
    return shavar_hash_ex(msg, nbits, shavar_iv, SHAVAR_ROUNDS, out);
  }
```

This code is used in section 37.

**40. Proof of work.** A hash function used for *proof of work*—PoW, a search for an input whose digest is numerically small—is not asked what the digest is but whether it is small enough. That question needs the 32 digest bytes read as one 256-bit integer, and **the order in which they are read is a convention, not a fact about the digest**. This section implements the Bitcoin convention, specified normatively in `spec/SPEC.md` §10.

Two terms before they are used. A *target* is the threshold, an unsigned 256-bit integer; the PoW is met when the digest's value is at most the target. *nBits* is a compact 32-bit encoding of a target, an 8-bit exponent above a 23-bit mantissa, and it is the form a Bitcoin block header carries. Note that the parameter named *nbits* in this section is that compact target and has nothing whatever to do with the message bit length called *nbits* everywhere else in this web. The collision of names is inherited from both conventions rather than chosen here, and it is the reason this paragraph exists.

**41. The byte order, which is the whole difficulty.** Let the digest bytes be numbered in *emission order*: $digest[0]$ is the first byte the hash function produced, the most significant byte of $H[0]$. The value compared against the target is

$$\text{value} \;=\; \sum_{i=0}^{31} digest[i] \cdot 256^{\,i}.$$

So $digest[0]$ is the **least** significant byte and $digest[31]$ is the **most** significant—the reverse of the order the bytes are written in.

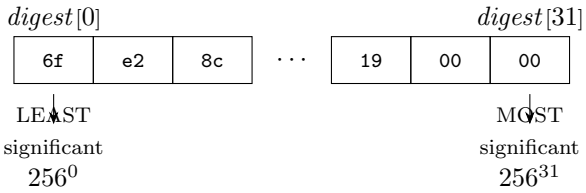


*Figure 4.* The little-endian reading of the digest. The bytes shown are the first and last of the Bitcoin genesis block's doubled SHA-256, in emission order. Read this way the value is `0x00000000 0019d668 ...`, which is the block hash as it is universally quoted; read the other way it would be `0x6fe28c0a ...`, about $2^{216}$ times larger, which fails every target ever used. This is why a block hash is *displayed* with its bytes reversed relative to the digest actually computed.

**42. Decoding nBits.** With *exponent* the high byte and *mantissa* the low 23 bits, the target is *mantissa* shifted left by $8(\mathit{exponent} - 3)$ bits, or shifted right by $8(3 - \mathit{exponent})$ when *exponent* is at most 3. The exponent counts *bytes*, not bits.

Bit `0x00800000` is a sign bit. A target is an unsigned magnitude, so a set sign bit is an error rather than something to mask away. An encoding is invalid, and must be reported as such rather than answered, when it is negative, when the target would not fit in 256 bits, or when the target is zero—nothing can be at or below zero, so such a request is unsatisfiable rather than merely unmet. The three overflow tests are each conditioned on a nonzero mantissa, because a zero mantissa is already caught by the zero-target test. This matches Bitcoin's `SetCompact` exactly.

No bignum arithmetic appears anywhere, here or in the six sibling implementations, because the repository's rule is core language only. The target is instead built as 32 bytes in *big*-endian order, and at most three mantissa bytes are placed into an otherwise zeroed array.

⟨ Proof of work 42 ⟩ ≡
  ⟨ Decode a compact target 43 ⟩
  ⟨ Compare a digest against a target 44 ⟩

This code is used in section 15.

**43.** *shift* is the byte offset of the mantissa's low byte from the least significant end, so in a big-endian array of 32 bytes that byte lands at index $31 - \mathit{shift}$ and the two above it at $30 - \mathit{shift}$ and $29 - \mathit{shift}$. The overflow test has already guaranteed that no nonzero byte would land at a negative index; the guards are kept anyway so that this section is correct read on its own.

⟨ Decode a compact target 43 ⟩ ≡

```
int shavar_pow_target(uint32_t nbits, unsigned char target[SHAVAR_DIGEST_BYTES])
{
  uint32_t exponent = nbits ≫ 24;
  uint32_t mantissa = nbits & #007FFFFFU;
  int shift;
  int i;
  if (mantissa ≠ 0U ∧ (nbits & #00800000U) ≠ 0U) return −1;
  if (mantissa ≠ 0U ∧ (exponent > 34U ∨ (mantissa > #FFU ∧ exponent > 33U) ∨ (mantissa >
      #FFFFU ∧ exponent > 32U))) {
    return −1;
  }
  for (i = 0; i < SHAVAR_DIGEST_BYTES; i++) target[i] = 0U;
  if (exponent ≤ 3U) {
    uint32_t v = mantissa ≫ (8U ∗ (3U − exponent));
    target[31] = (unsigned char)(v & #FFU);
    target[30] = (unsigned char)((v ≫ 8) & #FFU);
    target[29] = (unsigned char)((v ≫ 16) & #FFU);
  }
  else {
    shift = (int) exponent − 3;
    if (31 − shift ≥ 0) target[31 − shift] = (unsigned char)(mantissa & #FFU);
    if (30 − shift ≥ 0) target[30 − shift] = (unsigned char)((mantissa ≫ 8) & #FFU);
    if (29 − shift ≥ 0) target[29 − shift] = (unsigned char)((mantissa ≫ 16) & #FFU);
  }
  for (i = 0; i < SHAVAR_DIGEST_BYTES; i++) {
    if (target[i] ≠ 0U) return 0;
  }
  return −1;
}
```

This code is used in section 42.

**44.** The comparison walks both values most significant byte first and stops at the first difference. The target array is already in that order; the digest is not, so it is indexed backwards.

The expression *digest*[SHAVAR_DIGEST_BYTES − 1 − *i*] carries the entire convention of this section. Writing *digest*[*i*] there instead is the byte-order bug that §10.1 of the specification exists to prevent, and it is silent: it compiles without a warning, it runs, and it returns a plausible verdict that is wrong almost always. The test suite pins it with a pair of vectors that are the same 32 bytes in opposite orders and that disagree.

Reaching the end means every byte was equal, so the value equals the target. That is a pass: the relation is at-most, not strictly-less.

⟨ Compare a digest against a target 44 ⟩ ≡

```
int shavar_pow_check(const unsigned char digest[SHAVAR_DIGEST_BYTES], uint32_t nbits)
{
  unsigned char target[SHAVAR_DIGEST_BYTES];
  int i;
  if (shavar_pow_target(nbits, target) ≠ 0) return −1;
  for (i = 0; i < SHAVAR_DIGEST_BYTES; i++) {
    unsigned char a = digest[SHAVAR_DIGEST_BYTES − 1 − i];
    unsigned char b = target[i];
    if (a < b) return 1;
    if (a > b) return 0;
  }
  return 1;
}
```

This code is used in section 42.

**45. What this is not.** There is no mining loop: the comparison takes a digest that has already been computed, and searching for a satisfying input is the caller's business. There is no `powLimit` check either—Bitcoin rejects any target above a chain-specific maximum before checking a header, but that is a consensus parameter rather than a property of the encoding. And the floating-point *difficulty* ratio is deliberately absent: several of the seven languages have no native form for it, the shell implementation has no floating-point arithmetic at all, and seven independently rounded approximations of one ratio is exactly the kind of unstated boundary that the repository's cross-testing exists to keep out.

**46. The command-line driver.** This driver implements the uniform command-line contract that every implementation in the repository obeys byte for byte, specified in `spec/CLI.md`. The contract is rigid on purpose: cross-testing seven implementations is only as good as the comparison, and one format compared with `diff` keeps the harness trivial and honest.

It is kept in a separate file from the library so that the library object contains no input, no output and no global state, and so the static and shared libraries can be built from the library file alone.

Like the library, the driver calls no allocator. The message buffer is a single static array, which bounds the maximum message but makes the memory-safety story trivial to state: there is nothing to leak and nothing to free twice.

⟨ `main.c` 46 ⟩ ≡
⟨ Driver inclusions 47 ⟩
⟨ The message buffer 48 ⟩
⟨ Hexadecimal and output helpers 49 ⟩
⟨ Argument parsing 50 ⟩
⟨ The hash subcommand 54 ⟩
⟨ The trace subcommand 55 ⟩
⟨ Known-answer vectors 57 ⟩
⟨ The selftest subcommand 58 ⟩
⟨ Usage and main 63 ⟩
This code is cited in section 13.

**47.** ⟨ Driver inclusions 47 ⟩ ≡
```
#include "shavar.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```
This code is used in section 46.

**48.** One mebibyte, which is enough for the classic one-million-character known-answer vector.

⟨ The message buffer 48 ⟩ ≡
```
#define MSGMAX (1U << 20)
  static unsigned char g_msg[MSGMAX];
```
This code is used in section 46.

**49.** *unhex* decodes a hex string into bytes; the literal `-` means an empty message, since a zero-length argument would be indistinguishable from a missing one. It returns the byte count, or $-1$ on malformed input or overflow of the buffer.

```
⟨ Hexadecimal and output helpers 49 ⟩ ≡
  static int hexval(int c)
  {
    if (c ≥ '0' ∧ c ≤ '9') return c − '0';
    if (c ≥ 'a' ∧ c ≤ 'f') return c − 'a' + 10;
    if (c ≥ 'A' ∧ c ≤ 'F') return c − 'A' + 10;
    return −1;
  }
  static long unhex(const char *hex, unsigned char *out, size_t cap)
  {
    size_t n, i;
    if (strcmp(hex, "-") ≡ 0) return 0;
    n = strlen(hex);
    if (n % 2U ≠ 0U) return −1;
    if (n/2U > cap) return −1;
    for (i = 0; i < n; i += 2) {
      int hi = hexval((unsigned char) hex[i]);
      int lo = hexval((unsigned char) hex[i + 1]);
      if (hi < 0 ∨ lo < 0) return −1;
      out[i/2] = (unsigned char)((hi ≪ 4) | lo);
    }
    return (long)(n/2U);
  }
  static void put_digest(const unsigned char d[32])
  {
    int i;
    for (i = 0; i < 32; i++) printf("%02x", d[i]);
    putchar('\n');
  }
```

This code is used in section 46.

**50.**
```
⟨ Argument parsing 50 ⟩ ≡
  ⟨ Parsing a decimal integer 51 ⟩
  ⟨ Parsing the round count 52 ⟩
  ⟨ Checking that the two lengths agree 53 ⟩
```

This code is used in section 46.

**51.** A decimal 64-bit parse that rejects an empty string, any non-digit, and overflow. Rejecting overflow matters more than it looks: the bit count is allowed to be enormous, so silently wrapping would turn an absurd request into a plausible-looking answer.

```
⟨ Parsing a decimal integer 51 ⟩ ≡
  static int parse_u64(const char *s, uint64_t *out)
  {
    uint64_t v = 0;
    if (*s == '\0') return -1;
    for ( ; *s; s++) {
      if (*s < '0' || *s > '9') return -1;
      if (v > (UINT64_MAX - (uint64_t)(*s - '0'))/10U) return -1;
      v = v * 10U + (uint64_t)(*s - '0');
    }
    *out = v;
    return 0;
  }
```

This code is used in section 50.

**52.** The valid range for the round count is $0 \ldots 64$ inclusive, and it is enforced here rather than clamped in ⟨ The compression function 27 ⟩.

This is the second thing in this program that was changed after the fact, and the reason is worth recording alongside the rotation of ⟨ Word primitives 23 ⟩. An earlier version used *atoi* and let *shavar_compress* clamp, which meant that `hash <msg> <n> 100` printed a perfectly good SHA-256 digest in answer to a request that means nothing. Confidently wrong output of that kind is worse than an error message. Rejecting is the only honest response.

The boundary is also where independent implementations of the same specification diverged: one of the seven rejected a round count of zero, and this one clamped anything above 64, because `spec/CLI.md` did not originally say. Both readings were defensible; the specification now states the range, and the harness tests both ends of it.

Because *parse_u64* rejects anything that is not a run of digits, this also rejects `-1` and `12x` without a separate test.

```
⟨ Parsing the round count 52 ⟩ ≡
  static int parse_rounds(const char *s, int *out)
  {
    uint64_t v;
    if (parse_u64(s, &v) != 0) return -1;
    if (v > (uint64_t)SHAVAR_ROUNDS) return -1;
    *out = (int)v;
    return 0;
  }
```

This code is cited in section 28.

This code is used in section 50.

**53.** The hex argument and the bit count are two independent statements about the same message, so they have to be checked against each other: the byte count must be exactly $\lceil nbits/8 \rceil$.

```
⟨ Checking that the two lengths agree 53 ⟩ ≡
  static int lengths_agree(long nbytes, uint64_t nbits)
  {
    uint64_t need = nbits/8U + ((nbits % 8U) ? 1U : 0U);
    return (uint64_t) nbytes ≡ need;
  }
```

This code is used in section 50.

**54.** `hash` prints 64 lowercase hex digits and a newline, and nothing else. Every failure path returns 2 and writes to standard error, so that standard output is always either a well-formed digest or empty. That property is what lets the test harness compare outputs with `diff` instead of parsing them.

```
⟨ The hash subcommand 54 ⟩ ≡
  static int cmd_hash(const char *hex, const char *bits, const char *roundstr)
  {
    unsigned char digest[32];
    uint64_t nbits;
    long nbytes;
    int rounds = SHAVAR_ROUNDS;
    if (parse_u64(bits, &nbits) ≠ 0) {
      fprintf(stderr, "shavar: bad bit count '%s'\n", bits);
      return 2;
    }
    nbytes = unhex(hex, g_msg, MSGMAX);
    if (nbytes < 0) {
      fprintf(stderr, "shavar: malformed hex, or message exceeds %u bytes\n", MSGMAX);
      return 2;
    }
    if (¬lengths_agree(nbytes, nbits)) {
      fprintf(stderr, "shavar: %ld hex bytes does not match %llu bits\n", nbytes, (unsigned long
          long) nbits);
      return 2;
    }
    if (roundstr ∧ parse_rounds(roundstr, &rounds) ≠ 0) {
      fprintf(stderr, "shavar: rounds must be 0..64, got '%s'\n", roundstr);
      return 2;
    }
    if (shavar_hash_ex(g_msg, nbits, shavar_iv, rounds, digest) ≠ 0) {
      fprintf(stderr, "shavar: nonzero trailing bits in final byte\n");
      return 2;
    }
    put_digest(digest);
    return 0;
  }
```

This code is used in section 46.

**55.** `trace` prints the whole interior of one block's compression as tab-separated records. This is the subcommand that makes the program useful for studying the function rather than only for hashing: a digest mismatch between two implementations says that something is wrong, whereas a trace mismatch says which round and which word.

Note the loop that runs every block up to and including *idx* rather than jumping straight to the requested one. A trace of block 3 is meaningless without the chaining value that blocks 0 through 2 leave behind.

The record order — all of `HIN`, then all of $W$, then $A$, $E$, `T1`, `T2`, then `HOUT` — is fixed by the contract. $W$ is always printed for all 64 entries even under a reduced round count, while $A$ and $E$ stop at $tr.rounds$, for the reason given in ⟨ Build the message schedule 29 ⟩.

```
⟨ The trace subcommand 55 ⟩ ≡
  static int cmd_trace(const char *hex, const char *bits, const char *idxstr, const char *roundstr)
  {
    shavar_trace tr;
    unsigned char block[64];
    uint32_t h[8];
    uint64_t nbits, idx = 0, nblocks, i;
    long nbytes;
    int rounds = SHAVAR_ROUNDS, t;
    if (parse_u64(bits, &nbits) ≠ 0) {
      fprintf(stderr, "shavar: bad bit count '%s'\n", bits);
      return 2;
    }
    nbytes = unhex(hex, g_msg, MSGMAX);
    if (nbytes < 0 ∨ ¬lengths_agree(nbytes, nbits)) {
      fprintf(stderr, "shavar: malformed message\n");
      return 2;
    }
    if (idxstr ∧ parse_u64(idxstr, &idx) ≠ 0) {
      fprintf(stderr, "shavar: bad block index\n");
      return 2;
    }
    if (roundstr ∧ parse_rounds(roundstr, &rounds) ≠ 0) {
      fprintf(stderr, "shavar: rounds must be 0..64, got '%s'\n", roundstr);
      return 2;
    }
    nblocks = shavar_padded_blocks(nbits);
    if (idx ≥ nblocks) {
      fprintf(stderr, "shavar: block %llu out of range (%llu blocks)\n", (unsigned long
          long) idx, (unsigned long long) nblocks);
      return 2;
    }
    for (t = 0; t < 8; t++) h[t] = shavar_iv[t];
    for (i = 0; i ≤ idx; i++) {
      if (shavar_padded_block(g_msg, nbits, i, block) ≠ 0) {
        fprintf(stderr, "shavar: nonzero trailing bits in final byte\n");
        return 2;
      }
      shavar_compress(h, block, rounds, &tr);
    }
    ⟨ Emit the trace records 56 ⟩
    return 0;
  }
```

This code is used in section 46.

**56.** ⟨ Emit the trace records 56 ⟩ ≡

```
for (t = 0; t < 8; t++) printf("HIN\t%d\t%08x\n", t, tr.h_in[t]);
for (t = 0; t < SHAVAR_ROUNDS; t++) printf("W\t%d\t%08x\n", t, tr.W[t]);
for (t = -4; t < tr.rounds; t++) printf("A\t%d\t%08x\n", t, shavar_A(&tr, t));
for (t = -4; t < tr.rounds; t++) printf("E\t%d\t%08x\n", t, shavar_E(&tr, t));
for (t = 0; t < tr.rounds; t++) printf("T1\t%d\t%08x\n", t, tr.T1[t]);
for (t = 0; t < tr.rounds; t++) printf("T2\t%d\t%08x\n", t, tr.T2[t]);
for (t = 0; t < 8; t++) printf("HOUT\t%d\t%08x\n", t, tr.h_out[t]);
```

This code is used in section 55.

**57.** The built-in vectors are the classic FIPS 180-2 examples plus the empty string. A null *msg* field means the one-million-character vector, which is too large to write out. The bit-oriented vectors are not inlined here: they come from the NIST CAVP set, there are 1025 of them, and they live in `tests/vectors/` where the cross-implementation harness reads them.

⟨ Known-answer vectors 57 ⟩ ≡

```
struct kat {
  const char *msg;
  const char *want;
};
static const struct kat kats[] = {{"",
    "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"},{"abc",
    "ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad"},
    {"abcdbcdecdefdefgefghfghighijhijkijkljklmklmnlmnomnopnopq",
    "248d6a61d20638b8e5c026930c3e6039a33ce45964ff2167f6ecedd419db06c1"},
    {"abcdefghbcdefghicdefghijdefghijkefghijklfghijklmghijklmnhijklmnoijklmnopjklmnop\
    qklmnopq""rlmnopqrsmnopqrstnopqrstu",
    "cf5b16a778af8380036ce59e7b0492370b249b11e8f07a51afac45037afee9d1"},{Λ,
    "cdc76e5c9914fb9281a1c7e284d73e67f1809a48a497200e046d39ccc7112cd0"},};
```

This code is used in section 46.

**58.** The self-test checks three separate things, and the second and third are there because the first would not catch a failure in them.

```
⟨ The selftest subcommand 58 ⟩ ≡
  static int cmd_selftest(void)
  {
    size_t i;
    int fails = 0;
    int skipped = 0;
    char got[65];
    ⟨ Decide whether to skip the long vector 59 ⟩
    ⟨ Check the known-answer vectors 60 ⟩
    ⟨ Check the padding arithmetic 61 ⟩
    ⟨ Check that trailing bits are rejected 62 ⟩
    if (fails ≡ 0) {
      printf("ok␣%u\n", (unsigned)(sizeof kats/sizeof kats[0]) − (unsigned) skipped);
      if (skipped) fprintf(stderr, "(SHAVAR_QUICK:␣skipped␣%d␣long␣vector(s))\n", skipped);
      return 0;
    }
    return 1;
  }
```

This code is used in section 46.

**59.** The environment variable `SHAVAR_QUICK` skips the million-character vector. This exists for the sanitizer builds rather than for convenience. Under AddressSanitizer with stack-use-after-return detection, the kilobyte-scale trace that *shavar_compress* builds on every call is relocated to a poisoned frame and re-poisoned on each of the 15625 blocks that vector requires, which turns milliseconds into minutes. The check is still worth running — just not against a megabyte. Every code path is still executed by the short vectors, so what is restricted is the input size, never the coverage.

It is an environment variable rather than a command-line flag so that the contract in `spec/CLI.md` — `selftest` takes no arguments — holds exactly and this implementation stays interchangeable with the other six.

```
⟨ Decide whether to skip the long vector 59 ⟩ ≡
  const char *quick = getenv("SHAVAR_QUICK");
```

This code is used in section 58.

**60.** ⟨ Check the known-answer vectors 60 ⟩ ≡

```
for (i = 0; i < sizeof kats/sizeof kats[0]; i++) {
  unsigned char digest[32];
  uint64_t nbits;
  size_t len, j;
  if (kats[i].msg ≡ Λ) {
    if (quick ∧ quick[0] ≡ '1') {
      skipped++;
      continue;
    }
    len = 1000000U;
    memset(g_msg, 'a', len);
  }
  else {
    len = strlen(kats[i].msg);
    memcpy(g_msg, kats[i].msg, len);
  }
  nbits = (uint64_t) len * 8U;
  if (shavar_hash(g_msg, nbits, digest) ≠ 0) {
    printf("FAIL vector %u: hash returned error\n", (unsigned) i);
    fails++;
    continue;
  }
  for (j = 0; j < 32; j++) sprintf(got + 2 * j, "%02x", digest[j]);
  got[64] = '\0';
  if (strcmp(got, kats[i].want) ≠ 0) {
    printf("FAIL vector %u\n  want %s\n  got  %s\n", (unsigned) i, kats[i].want, got);
    fails++;
  }
}
```

This code is used in section 58.

**61.** Block counts at the boundaries where it is easy to be off by one. A message of $L$ bits needs $L+1+64$ bits of room, so 447 bits still fits in one block and 448 does not.

⟨ Check the padding arithmetic 61 ⟩ ≡

```
{
  struct {
    uint64_t bits;
    uint64_t blocks;
  } pb[] = {{0, 1}, {1, 1}, {446, 1}, {447, 1}, {448, 2}, {511, 2}, {512, 2}, {959, 2}, {960, 3}};
  size_t n;
  for (n = 0; n < sizeof pb/sizeof pb[0]; n++) {
    uint64_t got_b = shavar_padded_blocks(pb[n].bits);
    if (got_b ≠ pb[n].blocks) {
      printf("FAIL padding: %llu bits -> %llu blocks, want %llu\n", (unsigned long
          long) pb[n].bits, (unsigned long long) got_b, (unsigned long long) pb[n].blocks);
      fails++;
    }
  }
}
```

This code is used in section 58.

**62.** Both directions of the rejection rule are checked, and that is the point of this fragment.

`0xb4` is 1011 0100: as a 5-bit message its top five bits are the message and its low three are 100, which is nonzero and therefore illegal. `0xb0` is 1011 0000, the same five-bit message correctly encoded, and must be accepted. Testing only the rejection would be passed by an implementation that rejects everything.

This test earns its place. An implementation that silently masked the trailing bits instead of refusing them would pass every digest comparison in the repository's harness — the corpus generator is careful to clear those bits — and only a check like this one would notice.

```
⟨ Check that trailing bits are rejected 62 ⟩ ≡
  {
    unsigned char bad[1] = {#b4};
    unsigned char good[1] = {#b0};
    unsigned char digest[32];
    if (shavar_hash(bad, 5, digest) ≡ 0) {
      printf("FAIL: nonzero trailing bits were accepted\n");
      fails++;
    }
    if (shavar_hash(good, 5, digest) ≠ 0) {
      printf("FAIL: valid 5-bit message was rejected\n");
      fails++;
    }
  }
```

This code is used in section 58.

**63.** Dispatch. Anything unrecognised prints the usage message and exits 2, which is the same code used for malformed input; the contract distinguishes only success, a failing self-test, and everything else.

```
⟨ Usage and main 63 ⟩ ≡
  static void usage(void)
  {
    fputs("usage: shavar hash  <hex|-> <nbits> [rounds]\n"
          "       shavar trace <hex|-> <nbits> [blockidx] [rounds]\n"
          "       shavar selftest\n", stderr);
  }
  int main(int argc, char **argv)
  {
    if (argc < 2) {
      usage();
      return 2;
    }
    if (strcmp(argv[1], "hash") ≡ 0 ∧ argc ≥ 4) {
      return cmd_hash(argv[2], argv[3], argc ≥ 5 ? argv[4] : Λ);
    }
    if (strcmp(argv[1], "trace") ≡ 0 ∧ argc ≥ 4) {
      return cmd_trace(argv[2], argv[3], argc ≥ 5 ? argv[4] : Λ, argc ≥ 6 ? argv[5] : Λ);
    }
    if (strcmp(argv[1], "selftest") ≡ 0 ∧ argc ≡ 2) {
      return cmd_selftest();
    }
    usage();
    return 2;
  }
```

This code is used in section 46.

**64. Testing.** A literate program has a failure mode that an ordinary one does not: the prose and the code can drift apart, and worse, the tangled program can drift away from the hand-written program it is supposed to reproduce. Neither is caught by reading. `cweb/test_cweb.sh` runs the checks below; it is the thing that makes this document a tested artefact rather than a plausible-looking one.

**65. Is it the same program?** This is the load-bearing question, and it is answered three ways, from strongest to broadest.

1. *Token-level identity.* Both the tangled sources and the hand-written sources in `c/` are run through the C preprocessor and then through a small tokeniser that preserves string and character literals exactly and normalises everything else. The two token streams must be identical. This catches any drift at all, including drift in code that no test happens to execute — a changed constant in an unreached branch, say. When it fails it prints the first differing token with its context.
2. *Observational identity.* The tangled binary and the binary built from `c/` are run against the same inputs and must agree exactly. Digests are compared over a sample of messages including sub-byte bit lengths; full traces are compared record for record, which pins down every intermediate word of the compression rather than only the output; reduced-round digests are compared at several round counts including the boundaries 0 and 64; and the error paths are compared on exit status, standard output and standard error.
3. *The real test suite.* The tangled binary is substituted for the C implementation in the repository's own harness and run against the NIST CAVP known-answer vectors — 1154 of them, 896 with a bit length that is not a multiple of 8 — and against the other six implementations, on digests and on per-round traces.

The three are not redundant. The first would pass if both programs were identically wrong; the third would pass if a difference existed in a path the corpus never reaches; the second sits in between.

**66. Is the document itself sound?** The typeset output is treated as a build artefact with tests, in the same way the repository's other document is. `cweave` must report no errors — in particular it must not report a section that is used and never defined, or defined and never used — `tex` must complete without an error, the resulting `shavar-cweb.pdf` must exist, and its rendered text must contain no unresolved reference. TeX renders one of those as a pair of question marks and then exits successfully, so a document can be "built" and still be quietly broken; the check looks for that pair in the finished PDF. (The pair is deliberately not written out here, since this page would then be the thing that failed the test.)

The index and the list of section names below are generated by `cweave` from the code itself, so they cannot fall out of date with it. Every identifier appears in the index with the number of every section that mentions it, and the section that defines it in italics.

## 67. Index.

⟨ Argument parsing 50 ⟩ Used in section 46.
⟨ Build the message schedule 29 ⟩ Cited in section 55. Used in section 27.
⟨ Check that trailing bits are rejected 62 ⟩ Used in section 58.
⟨ Check the known-answer vectors 60 ⟩ Used in section 58.
⟨ Check the padding arithmetic 61 ⟩ Used in section 58.
⟨ Checking that the two lengths agree 53 ⟩ Used in section 50.
⟨ Clamp the round count and record the incoming chaining value 28 ⟩ Used in section 27.
⟨ Compare a digest against a target 44 ⟩ Used in section 42.
⟨ Count the padded blocks 34 ⟩ Used in section 33.
⟨ Decide the value of one padded byte 36 ⟩ Used in section 35.
⟨ Decide whether to skip the long vector 59 ⟩ Used in section 58.
⟨ Decode a compact target 43 ⟩ Used in section 42.
⟨ Driver inclusions 47 ⟩ Used in section 46.
⟨ Emit the trace records 56 ⟩ Used in section 55.
⟨ Feed forward 32 ⟩ Used in section 27.
⟨ Hash from a given initial value 38 ⟩ Used in section 37.
⟨ Hash from the standard initial value 39 ⟩ Used in section 37.
⟨ Hashing 37 ⟩ Cited in section 3. Used in section 15.
⟨ Hexadecimal and output helpers 49 ⟩ Used in section 46.
⟨ Interface constants 19 ⟩ Used in section 18.
⟨ Interface declarations 21 ⟩ Used in section 18.
⟨ Known-answer vectors 57 ⟩ Used in section 46.
⟨ Library inclusions 16 ⟩ Used in section 15.
⟨ Padding 33 ⟩ Cited in sections 3 and 12. Used in section 15.
⟨ Parsing a decimal integer 51 ⟩ Used in section 50.
⟨ Parsing the round count 52 ⟩ Cited in section 28. Used in section 50.
⟨ Produce one padded block 35 ⟩ Used in section 33.
⟨ Proof of work 42 ⟩ Used in section 15.
⟨ Rotate right 24 ⟩ Used in section 23.
⟨ Run the recurrence 31 ⟩ Used in section 27.
⟨ Seed the two tracks 30 ⟩ Used in section 27.
⟨ Shift right 25 ⟩ Used in section 23.
⟨ The compression function 27 ⟩ Cited in sections 3 and 52. Used in section 15.
⟨ The constant tables 22 ⟩ Used in section 15.
⟨ The hash subcommand 54 ⟩ Used in section 46.
⟨ The message buffer 48 ⟩ Used in section 46.
⟨ The selftest subcommand 58 ⟩ Used in section 46.
⟨ The six round functions 26 ⟩ Cited in sections 2, 3, and 21. Used in section 15.
⟨ The trace structure 20 ⟩ Cited in section 27. Used in section 18.
⟨ The trace subcommand 55 ⟩ Used in section 46.
⟨ Usage and main 63 ⟩ Used in section 46.
⟨ Word primitives 23 ⟩ Cited in sections 3 and 52. Used in section 15.
⟨ `main.c` 46 ⟩ Cited in section 13.
⟨ `shavar.h` 18 ⟩ Cited in section 13.

# The shavar literate program

# SHA-256 as a two-dimensional order-4 recurrence

a CWEB presentation of the C99 reference implementation



This document is generated from `cweb/shavar-cweb.w` by `cweave`. The same source file is processed by `ctangle` into the C program that `cweb/test_cweb.sh` builds and tests.